

MiniFacebook

Developeri: Floria Răzvan - Marian, Jantea Raluca - Mihaela

1. Prezentare generală

Un REST API simplificat de tip Facebook, unde utilizatorii pot crea postări și adăuga comentarii la postări. Aplicația oferă operații CRUD (Create, Read, Update, Delete) pentru utilizatori, postări și comentarii.

2. Tehnologii utilizate

Limbaj de programare: Java 25

Framework: Spring Boot 4.0.4

Bază de date: PostgreSQL

Acces la baza de date: Spring Data JPA

ORM: Hibernate

Reducerea codului repetitiv: Lombok

Framework pentru unit-testing: Mockito

Maven: Tehnologie pentru gestionarea dependențelor

3. Arhitectură generală

Proiectul urmează o arhitectură pe straturi (layered architecture) cu separare a reponsabilităților. Fiecare strat are o singură responsabilitate, respectând principiile SOLID și făcând codul ușor de testat.

- **Controller** - Gestionează cererile și răspunsurile HTTP, definește endpoint-urile REST API. Primește JSON de la clienți și delegă către stratul de servicii.
- **Service** - Conține logica de business, validarea datelor de intrare și rezolvarea entităților (încărcarea entităților complete din baza de date când se primesc doar ID-uri).
- **Repository** - Gestionează accesul la baza de date prin Spring Data JPA. Extinde interfața CrudRepository oferind operații CRUD standard.
- **Entity** - Entități adnotate cu mapări Hibernate, reprezentând tabelele din baza de date.

Structura pachetelor:

- controller/ - UserController, PostController, CommentController
- service/ - UserService, PostService, CommentService
- repository/ - UserRepository, PostRepository, CommentRepository
- entity/ - User, Post, Comment

4. Modelul de date

4.1 User - reprezintă un utilizator al platformei MiniFacebook

- userId (Long) - Cheie primară, generată automat
- username (String) - Obligatoriu, unic
- password (String) - Obligatoriu
- email (String) - Obligatoriu, unic
- phoneNumber (String) - Obligatoriu, unic
- score (Double) - Implicit 0.0
- posts (List<Post>) - Lista postărilor autorului (OneToMany, cascade delete)
- comments (List<Comment>) - Lista comentariilor autorului (OneToMany, cascade delete)

4.2 Post - reprezintă o postare a unui utilizator

- postId (Long) - Cheie primară, generată automat
- title (String) - Obligatoriu
- text (String) - Obligatoriu
- imageUrl (String) - Opțional
- author (User) - Relație ManyToOne
- creationDate (Instant) - Generată automat la creare prin @CreationTimestamp
- comments (List<Comment>) - Lista comentariilor postării (OneToMany, cascade delete)

4.3 Comment - reprezintă un comentariu adăugat unei postări

- commentId (Long) - Cheie primară, generată automat
- text (String) - Obligatoriu
- creationDate (Instant) - Generată automat la creare prin @CreationTimestamp
- author (User) - Relație ManyToOne

- post (Post) - Relație ManyToOne

5. Endpoint-uri

5.1 Utilizatori

- GET /users/getAll - Returnează toți utilizatorii
- GET /users/getById?id={id} - Returnează un utilizator după ID
- POST /users/insertUser - Creează un utilizator nou
- PUT /users/updateUser - Actualizează un utilizator existent
- DELETE /users/deleteById?userId={id} - Șterge un utilizator după ID

5.2 Postări

- GET /posts/getAll - Returnează toate postările
- GET /posts/getById?id={id} - Returnează o postare după ID
- GET /posts/getById?userId={id} - Returnează postările unui autor
- POST /posts/insertPost - Creează o postare nouă
- PUT /posts/updatePost - Actualizează o postare existentă
- DELETE /posts/deleteById?postId={id} - Șterge o postare după ID

5.3 Comentarii

- GET /comments/getAll - Returnează toate comentariile
- GET /comments/getById?id={id} - Returnează un comentariu după ID
- GET /comments/getByPostId?postId={id} - Returnează comentariile unei postări
- POST /comments/insertComment - Creează un comentariu nou
- PUT /comments/updateComment - Actualizează un comentariu existent
- DELETE /comments/deleteById?id={id} - Șterge un comentariu după ID

6. Exemple de request-uri si response-uri in Postman

The image displays three sequential screenshots of the Postman application interface, illustrating an API workflow.

First Screenshot (POST Request): The interface shows a POST request to `http://localhost:8080/users/insertUser`. The request body is raw JSON:

```
{  "username": "alex_popa",  "password": "parola123",  "email": "andrei@gmail.com",  "phoneNumber": "0753599373"}
```

. The status bar indicates a successful response: `200 OK` with a response time of `985 ms` and a body size of `281 B`.

Second Screenshot (GET Request): The interface shows a GET request to `http://localhost:8080/users/getAll`. The response body is raw JSON:

```
{  "userId": 5,  "username": "alex_popa",  "password": "parola123",  "email": "andrei@gmail.com",  "phoneNumber": "0753599373",  "score": 0.0}
```

. The status bar indicates a successful response: `200 OK` with a response time of `1.21 s` and a body size of `412 B`.

Third Screenshot (GET Request): The interface shows a GET request to `http://localhost:8080/users/getAll`. The response body is raw JSON:

```
[  {    "userId": 1,    "username": "foria_razvan",    "password": "parola123",    "email": "razvan@gmail.com",    "phoneNumber": "0753599378",    "score": 15.0  },  {    "userId": 5,    "username": "alex_popa",    "password": "parola123",    "email": "andrei@gmail.com",    "phoneNumber": "0753599373",    "score": 0.0  }]
```

. The status bar indicates a successful response: `200 OK` with a response time of `1.21 s` and a body size of `412 B`.

http://localhost:8080/posts/getByUserId?userId=999

POST http://localhost:8080/posts/insertPost

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "title": "Prima postare",
3   "text": "Uitati unde am fost azi!",
4   "imageUrl": "https://example.com/image.jpg",
5   "author": { "userId": 1 }
6 }
```

Body Cookies Headers (5) Test Results 200 OK 1.59 s 454 B

{ JSON Preview Visualize

```
1 {
2   "postId": 4,
3   "title": "Prima postare",
4   "text": "Uitati unde am fost azi!",
5   "imageUrl": "https://example.com/image.jpg",
6   "author": {
7     "userId": 1,
8     "username": "andreiutu",
9     "password": "parolaNoua123",
10    "email": "andrei1@gmail.com",
11    "phoneNumber": "0753599378",
12    "score": 0.0
13  }
14 }
```

Runner Start Proxy Cookies Vault Trash

PUT http://localhost:8080/posts/updatePost

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

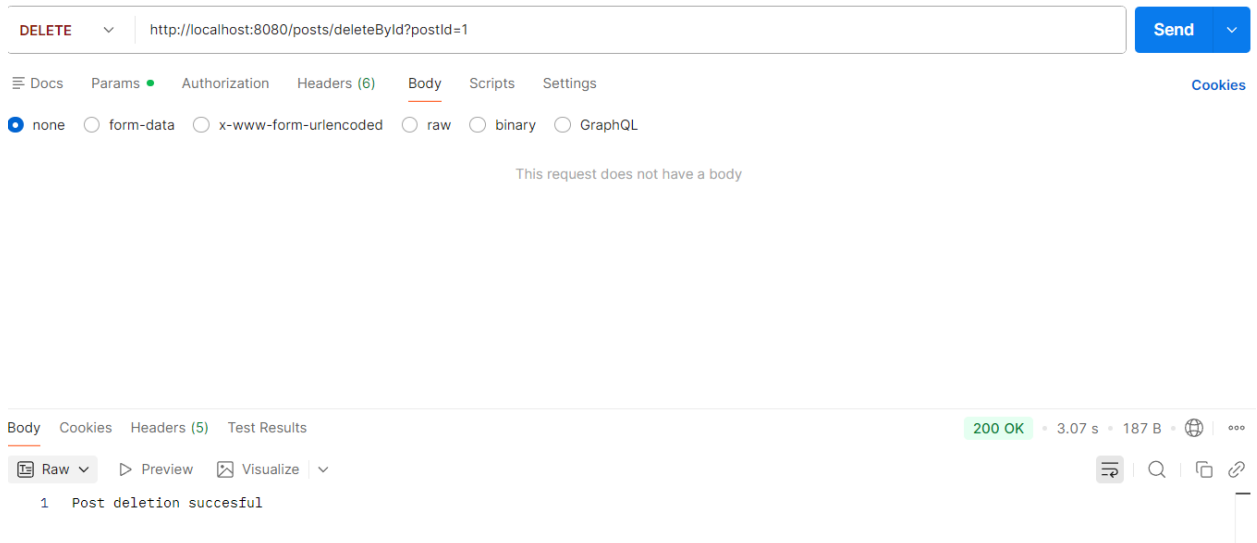
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "postId": 4,
3   "title": "Edited",
4   "text": "Am editat textul postarii.",
5   "imageUrl": "https://example.com/new-image.jpg",
6   "author": { "userId": 1 }
7 }
8
```

Body Cookies Headers (5) Test Results 200 OK 651 ms 453 B

{ JSON Preview Visualize

```
1 {
2   "postId": 4,
3   "title": "Edited",
4   "text": "Am editat textul postarii.",
5   "imageUrl": "https://example.com/new-image.jpg",
6   "author": {
7     "userId": 1,
8     "username": "andreiutu",
9     "password": "parolaNoua123",
10    "email": "andrei1@gmail.com",
11    "phoneNumber": "0753599378",
12    "score": 0.0
13  }
14 }
```



7. Unit Testing

Testele unitare sunt implementate folosind Mockito pentru toate cele trei clase de servicii:

- **UserServiceTest** - 8 teste: GET toți, GET după ID (găsit/negăsit), POST cu scor implicit, POST cu scor specificat, PUT, DELETE reușit, DELETE negăsit
- **PostServiceTest** - 10 teste: GET toate, GET după ID (găsit/negăsit), GET după ID utilizator (găsit/gol), POST cu autor, PUT păstrează data creării, PUT actualizează autorul autorul, ștergere reușită, ștergere negăsit
- **CommentServiceTest** - 9 teste: GET toate, GET după ID (găsit/negăsit), GET după ID postare (găsit/negăsit), inserare cu autor și postare, PUT păstrează data creării, DELETE id găsit, DELETE ID negăsit